



EST160 - Aula 04

Lucas Sousa

1 de Setembro de 2026

Conteúdos

Inspeção

Alguns Problemas

Armazenamento

Introdução

O nosso primeiro passo foi entender que tipos de dados podemos ler e analisar, embora não tenhamos entrado em muitos detalhes sobre todas as possibilidades.

Com os dados em mãos, agora precisamos realizar o que chamamos de **inspeção**, que é uma primeira avaliação desse conjunto de dados, e que vai nos ajudar a entender sua estrutura, e que ações precisaremos tomar para cumprir nossos objetivos.

Inspecionando os Dados

Ao inspecionar os dados, podemos utilizar as funções que já conhecemos, como `head()`, `tail()`, `str()`, `View()`, além de funções dos pacotes que vamos utilizar:

- ▶ `spec()`
- ▶ `problems()`
- ▶ `guess_encoding()`, `parse_logical()`, `parse_integer()`,
`parse_double()`, `parse_number()`, `parse_character()`,
`parse_factor()`, `parse_datetime()`, `parse_date()`,
`parse_time()`
- ▶ `glimpse()`, `dim()`, `nrow()`, `ncol()`
- ▶ `names()`, `class()`, `typeof()`
- ▶ `is.character()`, `is.numeric()`, `is.integer()`,
`is.logical()`, `is.factor()`, `is.Date()`

typeof() vs class() (vs mode())

Existe uma questão em como o R interage com objetos que o distingue de outras linguagens. Símbolos também são tratados como objetos. Temos uma distinção no que as funções `typeof()` e `class()` nos retornam.

- ▶ `class()` descreve a classe/comportamento do objeto
- ▶ `typeof()` descreve o tipo interno desse objeto para o R
- ▶ `mode()` descreve o objeto conforme apresentado na linguagem S

Mais detalhes podem ser vistos nesse link.

```
> x <- as.Date("2026-08-29")
> typeof(x)
[1] "double"
> class(x)
[1] "Date"
```

Coerção de Tipo

Um conceito importante é de **coerção de tipo**. Chamamos de *coerção* (coercion) a mudança de tipo de alguma variável.

Esse tipo de mudança pode ocorrer de automaticamente ou manualmente, o que chamaremos de coerção **implícita** ou **explícita**.

Coerção implícita acontece quando utilizamos um vetor em um contexto específico que espera um certo tipo de vetor.

Ex.: quando usamos um vetor lógico com uma função de soma

Coerção explícita acontece quando utilizamos uma função com a intenção de mudar o tipo da variável

Ex.: `as.logical()`, `as.integer()`, `as.double()`, ou `as.character()`

Coerção de Tipo

Na coerção implícita, o tipo mais complexo sempre vence.

```
typeof(c(TRUE, 1L))  
#> [1] "integer"
```

```
typeof(c(1L, 1.5))  
#> [1] "double"
```

```
typeof(c(1.5, "a"))  
#> [1] "character"
```

Coerção de Tipo

Temos a ordem

Lógico → Inteiro → Double → Character

Coerção de Tipo

Temos a ordem

Lógico → Inteiro → Double → Character

Para explícita, devemos ter cuidado.

```
> as.numeric(c(TRUE, 1L))  
[1] 1 1
```

```
> as.numeric(c(1.5, "a"))  
[1] 1.5 NA  
Mensagem de aviso:  
NAs introduzidos por coerção
```

Parsing

O slide anterior nos mostra que a coerção de tipo não é sempre a solução automática. Para evitar NA, vamos utilizar o que chamamos de **parsing**.

Parsing é o processo de analisar strings ou símbolos segundo determinadas regras para determinar sua estrutura ou significado.

No nosso contexto, estamos interpretando texto para produzir valores de tipos apropriados.

Parsing

O slide anterior nos mostra que a coerção de tipo não é sempre a solução automática. Para evitar NA, vamos utilizar o que chamamos de **parsing**.

Parsing é o processo de analisar strings ou símbolos segundo determinadas regras para determinar sua estrutura ou significado.

No nosso contexto, estamos interpretando texto para produzir valores de tipos apropriados.

Parsing

Nesse caso, podemos utilizar funções que vão interpretar o nosso texto como outro tipo de variável

```
str(parse_logical(c("TRUE", "FALSE", "NA")))
#>  logi [1:3] TRUE FALSE NA
```

```
str(parse_integer(c("1", "2", "3")))
#>   int [1:3] 1 2 3
```

```
str(parse_date(c("2010-01-01", "1979-10-14")))
#>   Date [1:2], format: "2010-01-01" "1979-10-14"
```

Parsing

Existem algumas estruturas para realizar o parsing. Podemos por argumentos para NA, e algumas tentativas podem nos retornar erros.

O pacote `{readr}` tenta adivinhar o tipo que devemos obter. Podemos utilizar a função `guess_parser()` para ver qual o melhor palpite do pacote, e `parse_guess()` irá utilizar esse palpite.

Parsing

Existem algumas estruturas para realizar o parsing. Podemos por argumentos para NA, e algumas tentativas podem nos retornar erros.

O pacote `{readr}` tenta adivinhar o tipo que devemos obter. Podemos utilizar a função `guess_parser()` para ver qual o melhor palpite do pacote, e `parse_guess()` irá utilizar esse palpite.

```
guess_parser("2010-10-01")  
#> [1] "date"  
guess_parser("15:01")  
#> [1] "time"  
guess_parser(c("TRUE", "FALSE"))  
#> [1] "logical"  
guess_parser(c("1", "5", "9"))  
#> [1] "double"  
guess_parser(c("12,352,561"))  
#> [1] "number"  
  
str(parse_guess("2010-10-10"))  
#> Date[1:1], format: "2010-10-10"
```

Coerção vs Parsing

Temos que frisar que embora seja processos parecidos, coerção de tipo e parsing **não são a mesma coisa**.

Coerção vs Parsing

Temos que frisar que embora seja processos parecidos, coerção de tipo e parsing **não são a mesma coisa**.

```
> as.numeric("R$ 1.234,56")  
[1] NA
```

Mensagem de aviso:

NAs introduzidos por coerção

Coerção vs Parsing

Temos que frisar que embora seja processos parecidos, coerção de tipo e parsing **não são a mesma coisa**.

```
> as.numeric("R$ 1.234,56")  
[1] NA
```

Mensagem de aviso:

NAs introduzidos por coerção

```
> parse_number(  
  "R$ 1.234,56",  
  locale = locale(decimal_mark = ",")  
)  
[1] 1234.56
```

if_else()

Podemos utilizar *parsing* com funções lógicas, como `dplyr::if_else()`.

A vantagem de `if_else()` (tidyverse) sobre `ifelse()` (nativa) é que ela conserva o tipo das variáveis.

```
> x <- factor(sample(letters[1:5], 10, replace =
  TRUE))
> x
[1] c e c d c a c d d c
Levels: a c d e
> ifelse(x %in% c("a", "b", "c"), x, NA)
[1] 2 NA 2 NA 2 1 2 NA NA 2
> if_else(x %in% c("a", "b", "c"), x, NA)
[1] c      <NA> c      <NA> c      a      c      <NA> <NA>
     > c
Levels: a c d e
```

if_else()

Podemos utilizar `if_else()` para lidar com alguns problemas ao realizar parsing.

```
library(dplyr)

x <- c(10, 20, 30, "40", 50, 60, "setenta", 80)
```

```
> parse_number(x)
Aviso: 1 parsing failure.
row col expected actual
  7  — a number setenta

[1] 10 20 30 40 50 60 NA 80
attr(,"problems")
# A tibble: 1 × 4
  row    col expected actual
  <int> <int> <chr>    <chr>
1     7    NA a number setenta
```

if_else()

Podemos utilizar `if_else()` para lidar com alguns problemas ao realizar parsing.

```
library(dplyr)

x <- c(10, 20, 30, "40", 50, 60, "setenta", 80)

> parse_number(x)
Aviso: 1 parsing failure.
row col expected actual
  7  — a number setenta

[1] 10 20 30 40 50 60 NA 80
attr(,"problems")
# A tibble: 1 × 4
  row    col expected actual
  <int> <int> <chr>    <chr>
1     7    NA a number setenta

> parse_number(if_else(x == "setenta", "70", x))
[1] 10 20 30 40 50 60 70 80
```

Renomeando colunas

É comum precisarmos renomear nossas colunas, principalmente se vamos ler uma base de dados não padronizada.

Uma questão que pode ser enfrentada é as variáveis não estarem em um formato aceitado pelo R. Nesse caso, elas serão **não sintáticas**, e serão apresentadas entre crases

```
`nome de exemplo`
```

Um nome sintático deve consistir de letras, dígitos, . (pontos), e _ (underline), mas não podem começar com ponto ou underline. Além disso, não podem usar palavras reservadas pelo R, como TRUE, NULL, if, function, etc. Uma lista pode ser encontrada aqui, ou com ?Reserved.

Renomeando colunas

Observe que os nomes que tinham espaços se tornaram non-syntactic names.

```
students
```

```
#> # A tibble: 6 × 5  
#>   `Student ID` `Full Name` favourite.food mealPlan AGE  
#>   <dbl> <chr> <chr> <chr> <chr>  
#> 1 1 Sunil Huffmann Strawberry yoghurt Lunch only 4  
#> 2 2 Barclay Lynn French fries Lunch only 5  
#> 3 3 Jayendra Lyne N/A Breakfast and lunch 7  
#> 4 4 Leon Rossini Anchovies Lunch only <NA>  
#> 5 5 Chidiegwu Dunkel Pizza Breakfast and lunch five  
#> 6 6 Güvenç Attila Ice cream Lunch only 6
```

Renomeando colunas

Podemos renomear as colunas numa estrutura tidy.

```
students |>
  rename(
    student_id = `Student ID`,
    full_name = `Full Name`
  )
```

#> # A tibble: 6 × 5

#>	student_id	full_name	favourite.food	mealPlan	AGE
#>	<dbl>	<chr>	<chr>	<chr>	<chr>
#> 1	1	Sunil Huffmann	Strawberry yoghurt	Lunch only	4
#> 2	2	Barclay Lynn	French fries	Lunch only	5
#> 3	3	Jayendra Lyne	<NA>	Breakfast and lunch	7
#> 4	4	Leon Rossini	Anchovies	Lunch only	<NA>
#> 5	5	Chidiegwu Dunkel	Pizza	Breakfast and lunch	five
#> 6	6	Güvenç Attila	Ice cream	Lunch only	6

Renomeando colunas

Uma alternativa é utilizar o pacote {janitor}, que possui funções úteis. A função `clean_names()` põe o nome das variáveis em `snake_case`.

```
students |> janitor::clean_names()
```

```
#> # A tibble: 6 × 5
#>   student_id full_name      favourite_food meal_plan      age
#>   <dbl> <chr>      <chr>      <chr>      <chr>
#> 1         1 Sunil Huffmann Strawberry yoghurt Lunch only      4
#> 2         2 Barclay Lynn   French fries    Lunch only      5
#> 3         3 Jayendra Lyne   <NA>      Breakfast and lunch 7
#> 4         4 Leon Rossini    Anchovies    Lunch only    <NA>
#> 5         5 Chidiegwu Dunkel Pizza      Breakfast and lunch five
#> 6         6 Güvenç Attila    Ice cream    Lunch only      6
```

Mudando tipo de variáveis

Podemos mudar o tipo de variáveis com a função `mutate()`. Utilizando operadores pipe nós podemos fazer tudo de forma bastante resumida.

```
students |>
  janitor::clean_names() |>
  mutate(meal_plan = factor(meal_plan))
```

```
#> # A tibble: 6 × 5
#>   student_id full_name      favourite_food meal_plan      age
#>   <dbl> <chr>      <chr>      <fct>      <chr>
#> 1         1 Sunil Huffmann Strawberry yoghurt Lunch only      4
#> 2         2 Barclay Lynn   French fries   Lunch only      5
#> 3         3 Jayendra Lyne   <NA>          Breakfast and lunch 7
#> 4         4 Leon Rossini    Anchovies     Lunch only     <NA>
#> 5         5 Chidiegwu Dunkel Pizza         Breakfast and lunch five
#> 6         6 Güvenç Attila   Ice cream     Lunch only      6
```

Lidando com texto e número

Podemos utilizar combinações de funções, como `parse_number()` do pacote `{readr}`, que nos ajuda a interpretar texto como número quando necessário.

```
students <- students |>
  janitor::clean_names() |>
  mutate(
    meal_plan = factor(meal_plan),
    age = parse_number(if_else(age == "five", "5", age))
  )
```

students

```
#> # A tibble: 6 × 5
```

#>	student_id	full_name	favourite_food	meal_plan	age
#>	<dbl>	<chr>	<chr>	<fct>	<dbl>
#> 1	1	Sunil Huffmann	Strawberry yoghurt	Lunch only	4
#> 2	2	Barclay Lynn	French fries	Lunch only	5
#> 3	3	Jayendra Lyne	<NA>	Breakfast and lunch	7
#> 4	4	Leon Rossini	Anchovies	Lunch only	NA
#> 5	5	Chidiegwu Dunkel	Pizza	Breakfast and lunch	5
#> 6	6	Güvenç Attila	Ice cream	Lunch only	6

Controlando Tipos das Colunas

Comentamos anteriormente que arquivos CSV não possuem informação sobre o tipo de variáveis. Nesse caso, o pacote `{readr}`

`{readr}` usa uma heurística para entender o tipo das colunas. Por padrão, ele vai analisar cada coluna a partir de uma quantidade de linhas, respondendo as seguintes questões:

- ▶ A coluna contém apenas F, T, FALSE ou TRUE? Se sim, então o tipo é lógico;
- ▶ A coluna tem apenas números (como 1, -4.5, 5e6, Inf)? Se sim, é um número;
- ▶ A coluna bate com o padrão ISO8601 (padrão para datas)? Se sim, é uma data ou data-horário;
- ▶ Caso contrário, o tipo da coluna é string.

Controlando Tipo das Colunas

Podemos determinar os tipos das colunas ao ler os dados

```
readr::read_csv(  
  "dados.csv",  
  col_types = cols(  
    coluna1 = col_character(),  
    coluna2 = col_double(),  
    coluna3 = col_integer()  
  )  
)
```

Renomeando colunas

- ▶ `col_logical()` e `col_double()` lê lógicos e números reais;
- ▶ `col_integer()` lê números inteiros, que ocupam metade memória ocupada por doubles;
- ▶ `col_character()` lê strings. É útil porque é comum termos números representando códigos, como placa de trânsito, números de cartão de crédito, etc;
- ▶ `col_factor()`, `col_date()`, `col_datetime()` para fatores, datas e data-horário;
- ▶ `col_number()` ignora os valores não numéricos;
- ▶ `col_skip()` pula colunas.

Problemas comuns

- ▶ Delimitador incorreto (ponto e vírgula, etc)
- ▶ Nome de variáveis incorretos
- ▶ Linhas extra ou dados textuais
- ▶ Quebra de linhas entre observações
- ▶ Número de colunas inconsistente
- ▶ Valores faltantes
- ▶ Tipos inconsistentes
- ▶ Datas ambíguas
- ▶ Encoding

Problemas comuns

- ▶ Delimitador incorreto (ponto e vírgula, etc)
- ▶ Nome de variáveis incorretos
- ▶ Linhas extra ou dados textuais
- ▶ Quebra de linhas entre observações
- ▶ Número de colunas inconsistente
- ▶ Valores faltantes
- ▶ Tipos inconsistentes
- ▶ Datas ambíguas
- ▶ Encoding

Problemas comuns

Para datas, um exemplo

```
parse_date(  
  "01/02/2025",  
  format = "%d/%m/%Y"  
)
```

Problemas comuns

Para datas, um exemplo

```
parse_date(  
  "01/02/2025",  
  format = "%d/%m/%Y"  
)
```

Podemos usar a função `problems()`

```
dados  
# A tibble: 3 × 3  
  id      name  comment  
  <chr>   <chr> <chr>  
1 1      Ana   Gostou muito  
2 do produto NA     NA  
3 2      Bruno Não gostou  
> problems(dados)  
# A tibble: 1 × 5  
  row    col expected  actual  file  
  <int> <int> <chr>    <chr>  <chr>  
1     3     1 3 columns 1 columns C:/Users/Nome/Documents
```

Organização Inicial

Agora que sabemos alguns dos problemas que podemos ter e entendemos os conceitos básicos para lidar com eles, vamos listar algumas funções do tidyverse que podemos nos ajudar

Podemos traçar um paralelo com o que tentamos fazer utilizando as funções nativas do R, e vamos tentar focar em utilizar as estratégias que tenham mais eficiência de código.

Não existe **certo** ou **errado**. mas vamos tentar analisar criticamente quais as melhores estratégias para atingir determinado objetivo.

Organização Inicial

```
select()
```

```
# seleciona colunas
```

```
dados |>
```

```
  select(id, nome, idade, renda)
```

```
# seleciona colunas que começam com um prefixo
```

```
dados |>
```

```
  select(starts_with("idade"))
```

Organização Inicial

```
filter()
```

```
# Filtra com base em uma informação  
dados |>  
  filter(idade >= 18)
```

```
# Nativa do R
```

```
subset(dados, idade >= 18)
```

Organização Inicial

```
arrange()
```

```
# Organiza um vetor
```

```
dados |>  
  arrange(idade)
```

```
# Nativa do R
```

```
dados[order(dados$idade), , drop = FALSE]
```

Organização Inicial

```
mutate()
```

```
# Modifica uma coluna
```

```
dados |>
```

```
  mutate(
```

```
    renda_anual = renda_mensal * 12
```

```
)
```

```
# Nativa do R
```

```
dados$renda_anual <- dados$renda_mensal * 12
```

Organização Inicial

```
mutate()
```

```
# Modifica uma coluna
```

```
dados |>
```

```
  mutate(
```

```
    renda_anual = renda_mensal * 12
```

```
)
```

```
# Ou mais de uma
```

```
dados |>
```

```
  mutate(
```

```
    renda_anual = renda * 12,
```

```
    renda_milhares = renda_anual / 1000
```

```
)
```

```
# Nativa do R
```

```
dados$renda_anual <- dados$renda_mensal * 12
```

Organização Inicial

```
rename()
```

```
# Modifica nome de coluna  
dados |>  
  rename(  
    renda_mensal = renda  
  )
```

```
# Nativa do R  
names(dados)[names(dados) == "renda"] <- "renda_  
  mensal"
```

```
# Ou se tiver a posição
```

```
names(dados)[3] <- "renda_mensal"
```

Organização Inicial

```
relocate()
```

```
# Realoca colunas
```

```
dados |>
```

```
  relocate(id, .before = nome)
```

```
# Nativa do R
```

```
dados[c(
```

```
  "id",
```

```
  setdiff(names(dados), "id")
```

```
)]
```

Organização Inicial

```
distinct()
```

```
# Mostra quais são as observações distintas que  
  existem
```

```
dados |>  
  distinct()
```

```
# Nativa do R  
unique(dados)
```

Organização Inicial

```
slice()
```

```
# Pega uma parte dos dados  
dados |>  
  slice(1:10)
```

```
# Nativa do R  
dados[1:10, , drop = FALSE]
```

Organização Inicial

```
pull()
```

```
# Filtra colunas  
dados |>  
  pull(idade)
```

```
# Nativa do R  
dados$idade
```

Organização Inicial

group_by()

```
> dados
# A tibble: 5 × 2
  cidade      idade
  <chr>      <dbl>
1 Viçosa      20
2 Viçosa      25
3 Belo Horizonte 30
4 Belo Horizonte 40
5 Viçosa      35

> grupo <- dados |> group_by(cidade)
> grupo
# A tibble: 5 × 2
# Groups:   cidade [2]
  cidade      idade
  <chr>      <dbl>
1 Viçosa      20
2 Viçosa      25
3 Belo Horizonte 30
4 Belo Horizonte 40
5 Viçosa      35

> class(grupo)
[1] "grouped_df" "tbl_df"      "tbl"         "data.frame"
```

Aqui a separação se tornou **propriedade** do nosso tibble (Observe a linha Groups: cidade [2]). Não há equivalente nativo do R.

Organização Inicial

```
summarise()
```

```
# Calcula medidas resumo colunas  
dados |>  
  summarise(  
    media = mean(idade, na.rm = TRUE),  
  )
```

```
# Média para grupos
```

```
dados |>  
  group_by(sexo) |>  
  summarise(  
    media = mean(idade, na.rm = TRUE)  
  )
```

```
# Nativa do R  
mean(dados$idade, na.rm = TRUE)
```

Organização Inicial

`left_join()`, `inner_join()`, `full_join()`, `anti_join()`, `semi_join()`

Função	Descrição
<code>left_join()</code>	Mantenha todas as linhas da tabela da esquerda e acrescente as correspondências da direita.
<code>inner_join()</code>	Mantenha somente as linhas que possuem correspondência nas duas tabelas.
<code>full_join()</code>	Mantenha todas as linhas das duas tabelas.
<code>anti_join()</code>	Mantenha as linhas da primeira tabela que não possuem correspondência na segunda.
<code>semi_join()</code>	Mantenha as linhas da primeira tabela que possuem correspondência na segunda.

Chequem sempre as funções usando ?

Nativa do R

```
merge(  
  pessoas ,  
  municipios ,  
  by = "cod_municipio",  
  all.x = TRUE  
)
```

Organização Inicial

`pivot_longer()` e `pivot_wider()`

```
# Formato wide
```

id	janeiro	fevereiro	março
1	10	20	30
2	15	25	35

```
# Mudando para longer
```

```
dados |>  
  pivot_longer(  
    cols = janeiro:março,  
    names_to = "mes",  
    values_to = "vendas"  
  )
```

```
> dados
```

```
# A tibble: 6 × 3
```

	id	mes	vendas
	<int>	<chr>	<dbl>
1	1	janeiro	10
2	1	fevereiro	20
3	1	março	30
4	2	janeiro	15
5	2	fevereiro	25
6	2	março	35

Organização Inicial

`pivot_longer()` e `pivot_wider()`

```
# Nativa do R
reshape(
  dados,
  varying = c("janeiro", "fevereiro", "março"),
  v.names = "vendas",
  timevar = "mes",
  times = c("janeiro", "fevereiro", "março"),
  direction = "long"
)
```

Armazenamento

O GitHub limita o armazenamento a arquivos de até 100mb (veja detalhes aqui).

Para armazenar arquivos maiores, eles oferecem Git Large File Storage.

A questão do armazenamento precisa ser pensada com cuidado. Lembre-se que a nuvem é só o computador de outra pessoa (ou um pouco mais que isso).

Além disso, leva-se em consideração a privacidade, caso o projeto contenha dados sensíveis. Veja a documentação sobre dados sensíveis aqui.

Armazenamento

Avaliem com cuidado a possibilidade de utilizar o armazenamento institucional de vocês. Vejam

- ▶ Até quando perderão acesso
- ▶ Os termos desses armazenamentos
- ▶ O que pode mudar nessas condições
- ▶ Se a empresa tem direito de apagar ou tirar acesso a seus dados
- ▶ Que tipo de problemas ocorreram no passado

Avaliem com cuidado serviços de armazenamento, e também procure opções open source ou alternativas quando fizer sentido.

- ▶ Mantenha mais de uma cópia dos dados
- ▶ Não altere ou apague os dados brutos

Armazenamento

Como e onde iremos armazenar os dados depende da nossa necessidade.

Para pesquisas e alguns tipos de projetos, é muito importante que os dados estejam acessíveis para que possamos ter reprodutibilidade e replicabilidade.

Além disso, pode ser necessário manter os dados de forma mais acessíveis, se eles forem muito difíceis de acessar.

Outros conceitos vão surgir futuramente, como *data lakes* e *data warehouses*